

120+ Tricky Java Guess-the-Output Questions (with Explanations)

Curated from common Java interview output patterns (MCQ + execution-order + language gotchas).

Section 1: Primitives, Promotion, and Operators (Q1-15)

Q1

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println('b' + 'i' + 't');  
    }  
}
```

Answer: 319 **Explanation:** Char literals are promoted to int and summed.

Q2

```
public class Main {  
    public static void main(String[] args) {  
        int a = 5;  
        System.out.println(a++ + ++a);  
    }  
}
```

Answer: 12 **Explanation:** a++ gives 5 then a becomes 6, ++a makes 7.

Q3

```
public class Main {  
    public static void main(String[] args) {  
        byte b = 10;  
        b = (byte) (b + 1);  
        System.out.println(b);  
    }  
}
```

Answer: 11

Q4

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(10 + 20 + "30");  
    }  
}
```

```
}  
}
```

Answer: 3030

Q5

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("10" + 20 + 30);  
    }  
}
```

Answer: 102030

Q6

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(10 + 20 + "30" + 40 + 50);  
    }  
}
```

Answer: 30304050

Q7

```
public class Main {  
    public static void main(String[] args) {  
        int x = 1;  
        x += 2.5;  
        System.out.println(x);  
    }  
}
```

Answer: 3 **Explanation:** Compound assignment includes implicit cast.

Q8

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(5 / 2);  
        System.out.println(5 / 2.0);  
    }  
}
```

Answer: 2 then 2.5

Q9

```
public class Main {  
    public static void main(String[] args) {  
        int a = 1;  
        int b = a++ + ++a + a;  
        System.out.println(b);  
    }  
}
```

Answer: 6

Q10

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(true ? "A" : 1);  
    }  
}
```

Answer: A

Q11

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(false ? "A" : 1);  
    }  
}
```

Answer: 1

Q12

```
public class Main {  
    public static void main(String[] args) {  
        int x = 10;  
        System.out.println(x > 5 ? x < 20 ? "Y" : "N" : "Z");  
    }  
}
```

Answer: Y

Q13

```
public class Main {  
    public static void main(String[] args) {
```

```
        System.out.println(~5);
    }
}
```

Answer: -6

Q14

```
public class Main {
    public static void main(String[] args) {
        System.out.println(1 << 3);
    }
}
```

Answer: 8

Q15

```
public class Main {
    public static void main(String[] args) {
        System.out.println(-8 >> 1);
        System.out.println(-8 >>> 1);
    }
}
```

Answer: -4 then 2147483644

Section 2: Strings, Pool, Equals, and Immutability (Q16-30)

Q16

```
public class Main {
    public static void main(String[] args) {
        String a = "go";
        String b = "go";
        System.out.println(a == b);
    }
}
```

Answer: true

Q17

```
public class Main {
    public static void main(String[] args) {
        String a = new String("go");
        String b = new String("go");
        System.out.println(a == b);
    }
}
```

```
}  
}
```

Answer: false

Q18

```
public class Main {  
    public static void main(String[] args) {  
        String a = new String("go");  
        String b = "go";  
        System.out.println(a.equals(b));  
    }  
}
```

Answer: true

Q19

```
public class Main {  
    public static void main(String[] args) {  
        String a = "go";  
        String b = a + "lang";  
        String c = "golang";  
        System.out.println(b == c);  
    }  
}
```

Answer: false

Q20

```
public class Main {  
    public static void main(String[] args) {  
        String b = "go" + "lang";  
        String c = "golang";  
        System.out.println(b == c);  
    }  
}
```

Answer: true **Explanation:** Compile-time constant folding.

Q21

```
public class Main {  
    public static void main(String[] args) {  
        String s = "abc";  
        s.concat("d");  
    }  
}
```

```
        System.out.println(s);
    }
}
```

Answer: abc

Q22

```
public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder("a");
        sb.append("b");
        System.out.println(sb);
    }
}
```

Answer: ab

Q23

```
public class Main {
    public static void main(String[] args) {
        String s = null;
        System.out.println(String.valueOf(s));
    }
}
```

Answer: null

Q24

```
public class Main {
    public static void main(String[] args) {
        String s = null;
        System.out.println(s + "x");
    }
}
```

Answer: nullx

Q25

```
public class Main {
    public static void main(String[] args) {
        String s = "abc";
        System.out.println(s.substring(1, 2));
    }
}
```

Answer: b

Q26

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello".charAt(4));  
    }  
}
```

Answer: o

Q27

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("abc".toUpperCase().toLowerCase());  
    }  
}
```

Answer: abc

Q28

```
public class Main {  
    public static void main(String[] args) {  
        String s = " a b ";  
        System.out.println "[" + s.trim() + "];"  
    }  
}
```

Answer: [a b]

Q29

```
public class Main {  
    public static void main(String[] args) {  
        String s = "java";  
        System.out.println(s.indexOf('v'));  
    }  
}
```

Answer: 2

Q30

```
public class Main {  
    public static void main(String[] args) {
```

```
String s = "java";
System.out.println(s.replace('a', 'o'));
}
}
```

Answer: jovo

Section 3: OOP, Inheritance, Overloading/Overriding (Q31-45)

Q31

```
class A { void f() { System.out.print("A"); } }
class B extends A { void f() { System.out.print("B"); } }
public class Main {
    public static void main(String[] args) {
        A x = new B();
        x.f();
    }
}
```

Answer: B

Q32

```
class A { static void f() { System.out.print("A"); } }
class B extends A { static void f() { System.out.print("B"); } }
public class Main {
    public static void main(String[] args) {
        A x = new B();
        x.f();
    }
}
```

Answer: A **Explanation:** Static methods are hidden, not overridden.

Q33

```
class A { A() { System.out.print("A"); } }
class B extends A { B() { System.out.print("B"); } }
public class Main {
    public static void main(String[] args) {
        new B();
    }
}
```

Answer: AB

Q34

```

class A {
    int x = 1;
}
class B extends A {
    int x = 2;
}
public class Main {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.x);
    }
}

```

Answer: 1 **Explanation:** Field access is compile-time by reference type.

Q35

```

class A {
    void f(Number n) { System.out.print("N"); }
    void f(Integer n) { System.out.print("I"); }
}
public class Main {
    public static void main(String[] args) {
        new A().f(10);
    }
}

```

Answer: I

Q36

```

class A {
    A() { this(1); System.out.print("A"); }
    A(int x) { System.out.print(x); }
}
public class Main {
    public static void main(String[] args) {
        new A();
    }
}

```

Answer: 1A

Q37

```

class A {
    final void f() { }
}
class B extends A {

```

```
// void f() {}  
}  
public class Main {}
```

Answer: If uncommented, compile error.

Q38

```
interface X { default void f() { System.out.print("X"); } }  
class A implements X {}  
public class Main {  
    public static void main(String[] args) {  
        new A().f();  
    }  
}
```

Answer: X

Q39

```
class A {  
    private void f() { System.out.print("A"); }  
    void g() { f(); }  
}  
class B extends A {  
    private void f() { System.out.print("B"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new B().g();  
    }  
}
```

Answer: A

Q40

```
class A {  
    void f(Object o) { System.out.print("O"); }  
    void f(String s) { System.out.print("S"); }  
}  
public class Main {  
    public static void main(String[] args) {  
        new A().f(null);  
    }  
}
```

Answer: S

Q41

```
class A {
    A() { System.out.print("A"); }
}
class B extends A {
    B() { System.out.print("B"); }
}
class C extends B {
    C() { System.out.print("C"); }
}
public class Main {
    public static void main(String[] args) { new C(); }
}
```

Answer: ABC

Q42

```
class A {
    static { System.out.print("S1"); }
    { System.out.print("I1"); }
    A() { System.out.print("C1"); }
}
public class Main {
    public static void main(String[] args) {
        new A(); new A();
    }
}
```

Answer: S1I1C1I1C1

Q43

```
class A {
    int f() { return 1; }
}
class B extends A {
    @Override int f() { return 2; }
}
public class Main {
    public static void main(String[] args) {
        A a = new B();
        System.out.println(a.f());
    }
}
```

Answer: 2

Q44

```
class A {  
    void f() throws Exception {}  
}  
class B extends A {  
    @Override void f() {}  
}  
public class Main {}
```

Answer: Compiles. **Explanation:** Overriding method can throw fewer/narrower checked exceptions.

Q45

```
class A {  
    A(String s) { }  
}  
class B extends A {  
    B() { super("x"); }  
}  
public class Main {  
    public static void main(String[] args) { new B(); }  
}
```

Answer: No output, compiles.

Section 4: Exceptions, finally, and Control Flow (Q46-60)

Q46

```
public class Main {  
    static int f() {  
        try { return 1; }  
        finally { return 2; }  
    }  
    public static void main(String[] args) {  
        System.out.println(f());  
    }  
}
```

Answer: 2

Q47

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int x = 10 / 0;  
        }  
    }  
}
```

```
        System.out.println(x);
    } catch (ArithmeticException e) {
        System.out.println("AE");
    }
}
}
```

Answer: AE

Q48

```
public class Main {
    public static void main(String[] args) {
        try {
            String s = null;
            s.length();
        } catch (NullPointerException e) {
            System.out.println("NPE");
        } finally {
            System.out.println("F");
        }
    }
}
```

Answer: NPE then F

Q49

```
public class Main {
    public static void main(String[] args) {
        System.out.println("A");
        if (true) return;
        // System.out.println("B");
    }
}
```

Answer: A

Q50

```
public class Main {
    public static void main(String[] args) {
        int i = 0;
        while (i++ < 3) {
            System.out.print(i);
        }
    }
}
```

Answer: 123

Q51

```
public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            if (i == 1) continue;
            System.out.print(i);
        }
    }
}
```

Answer: 02

Q52

```
public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 3; i++) {
            if (i == 1) break;
            System.out.print(i);
        }
    }
}
```

Answer: 0

Q53

```
public class Main {
    public static void main(String[] args) {
        int x = 1;
        switch (x) {
            case 1: System.out.print("A");
            case 2: System.out.print("B");
            default: System.out.print("C");
        }
    }
}
```

Answer: ABC **Explanation:** Fallthrough without break .

Q54

```
public class Main {
    public static void main(String[] args) {
        Integer a = 127, b = 127;
        System.out.println(a == b);
    }
}
```

```
}  
}
```

Answer: true

Q55

```
public class Main {  
    public static void main(String[] args) {  
        Integer a = 128, b = 128;  
        System.out.println(a == b);  
    }  
}
```

Answer: false **Explanation:** Integer cache usually -128 to 127.

Q56

```
public class Main {  
    public static void main(String[] args) {  
        Integer a = 10;  
        int b = 10;  
        System.out.println(a == b);  
    }  
}
```

Answer: true

Q57

```
public class Main {  
    public static void main(String[] args) {  
        int[] a = {1,2,3};  
        System.out.println(a.length);  
    }  
}
```

Answer: 3

Q58

```
public class Main {  
    public static void main(String[] args) {  
        int[] a = new int[2];  
        System.out.println(a[0] + " " + a[1]);  
    }  
}
```

Answer: 0 0

Q59

```
public class Main {  
    public static void main(String[] args) {  
        String[] a = new String[2];  
        System.out.println(a[0] == null);  
    }  
}
```

Answer: true

Q60

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println(Math.round(2.5));  
        System.out.println(Math.round(3.5));  
    }  
}
```

Answer: 3 then 4

Section 5: Threads, Runnables, Synchronization, and Virtual Threads (Q61-75)

Q61

```
class Main {  
    public static void main(String[] args) {  
        Thread t = new Thread(() -> System.out.print("T"));  
        t.run();  
        System.out.print("M");  
    }  
}
```

Answer: TM **Explanation:** run() executes on current thread, not a new thread.

Q62

```
class Main {  
    public static void main(String[] args) throws Exception {  
        Thread t = new Thread(() -> System.out.print("T"));  
        t.start();  
        t.join();  
        System.out.print("M");  
    }  
}
```

```
}  
}
```

Answer: TM

Q63

```
class Main {  
    public static void main(String[] args) {  
        Thread t = new Thread(() -> System.out.print("X"));  
        t.start();  
        System.out.print("Y");  
    }  
}
```

Answer: XY or YX **Explanation:** Scheduling is nondeterministic.

Q64

```
class Main {  
    static int c = 0;  
    public static void main(String[] args) throws Exception {  
        Thread t1 = new Thread(() -> { for (int i = 0; i < 1000; i++) c++; });  
        Thread t2 = new Thread(() -> { for (int i = 0; i < 1000; i++) c++; });  
        t1.start(); t2.start();  
        t1.join(); t2.join();  
        System.out.println(c);  
    }  
}
```

Answer: Not guaranteed to be 2000 **Explanation:** c++ is not atomic.

Q65

```
class Main {  
    static int c = 0;  
    static synchronized void inc() { c++; }  
    public static void main(String[] args) throws Exception {  
        Thread t1 = new Thread(() -> { for (int i = 0; i < 1000; i++) inc(); });  
        Thread t2 = new Thread(() -> { for (int i = 0; i < 1000; i++) inc(); });  
        t1.start(); t2.start();  
        t1.join(); t2.join();  
        System.out.println(c);  
    }  
}
```

Answer: 2000

Q66

```

class Main {
    public static void main(String[] args) throws Exception {
        Thread t = new Thread(() -> {
            try { Thread.sleep(50); } catch (InterruptedException e) { }
            System.out.print("A");
        });
        t.start();
        t.join();
        System.out.print("B");
    }
}

```

Answer: AB

Q67

```

class Main {
    public static void main(String[] args) {
        Runnable r = () -> System.out.print("R");
        r.run();
        System.out.print("M");
    }
}

```

Answer: RM

Q68

```

class Main {
    public static void main(String[] args) throws Exception {
        var t = Thread.ofVirtual().start(() -> System.out.print("V"));
        t.join();
        System.out.print("M");
    }
}

```

Answer: VM **Explanation:** Virtual thread (Java 21+) still needs join for deterministic ordering.

Q69

```

class Main {
    public static void main(String[] args) {
        Thread.ofVirtual().start(() -> System.out.print("V"));
        System.out.print("M");
    }
}

```

Answer: Usually MV or VM, may miss V if process exits quickly.

Q70

```
import java.util.concurrent.*;
class Main {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newSingleThreadExecutor();
        Future<Integer> f = ex.submit(() -> 40 + 2);
        System.out.println(f.get());
        ex.shutdown();
    }
}
```

Answer: 42

Q71

```
import java.util.concurrent.*;
class Main {
    public static void main(String[] args) throws Exception {
        ExecutorService ex = Executors.newFixedThreadPool(2);
        ex.execute(() -> System.out.print("A"));
        ex.execute(() -> System.out.print("B"));
        ex.shutdown();
    }
}
```

Answer: AB or BA

Q72

```
class Main {
    public static void main(String[] args) {
        System.out.println(Thread.currentThread().isDaemon());
    }
}
```

Answer: false **Explanation:** Main thread is non-daemon.

Q73

```
class Main {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {});
        System.out.println(t.getState());
    }
}
```

Answer: NEW

Q74

```
class Main {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {});
        t.start();
        try {
            t.start();
        } catch (IllegalThreadStateException e) {
            System.out.println("ITS");
        }
    }
}
```

Answer: ITS

Q75

```
class Main {
    static volatile boolean done = false;
    public static void main(String[] args) throws Exception {
        Thread t = new Thread(() -> done = true);
        t.start();
        t.join();
        System.out.println(done);
    }
}
```

Answer: true

Section 6: Collections and Generics Gotchas (Q76-90)

Q76

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        List<Integer> a = Arrays.asList(1,2,3);
        a.set(0, 9);
        System.out.println(a);
    }
}
```

Answer: [9, 2, 3]

Q77

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        List<Integer> a = Arrays.asList(1,2,3);
        try {
            a.add(4);
        } catch (UnsupportedOperationException e) {
            System.out.println("UOE");
        }
    }
}
```

Answer: UOE

Q78

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        List<Integer> a = List.of(1,2,3);
        System.out.println(a.contains(2));
    }
}
```

Answer: true

Q79

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Set<Integer> s = new HashSet<>();
        s.add(1); s.add(1);
        System.out.println(s.size());
    }
}
```

Answer: 1

Q80

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> m = new HashMap<>();
        m.put("a",1);
        m.put("a",2);
        System.out.println(m.get("a"));
    }
}
```

```
}  
}
```

Answer: 2

Q81

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        Map<String,Integer> m = new HashMap<>();  
        m.put(null, 10);  
        System.out.println(m.get(null));  
    }  
}
```

Answer: 10

Q82

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        Map<String,Integer> m = new TreeMap<>();  
        try {  
            m.put(null, 1);  
        } catch (NullPointerException e) {  
            System.out.println("NPE");  
        }  
    }  
}
```

Answer: NPE

Q83

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        List<Integer> a = new ArrayList<>(List.of(1,2,3));  
        for (Integer x : a) {  
            if (x == 2) a.remove(x);  
        }  
    }  
}
```

Answer: Throws ConcurrentModificationException .

Q84

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        List<Integer> a = new ArrayList<>(List.of(1,2,3));
        a.removeIf(x -> x % 2 == 0);
        System.out.println(a);
    }
}
```

Answer: [1, 3]

Q85

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        List<String> a = new ArrayList<>();
        a.add("x");
        a.add("y");
        Collections.reverse(a);
        System.out.println(a);
    }
}
```

Answer: [y, x]

Q86

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Queue<Integer> q = new LinkedList<>();
        q.offer(1); q.offer(2);
        System.out.println(q.poll() + " " + q.peek());
    }
}
```

Answer: 1 2

Q87

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Deque<Integer> d = new ArrayDeque<>();
        d.push(1);
        d.push(2);
        System.out.println(d.pop());
    }
}
```

```
}  
}
```

Answer: 2

Q88

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        List<Integer> a = List.of(1,2,3);  
        System.out.println(a.getClass().getSimpleName().contains("Immutable"));  
    }  
}
```

Answer: Often true , implementation detail.

Q89

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        Map<Integer,String> m = new LinkedHashMap<>();  
        m.put(2, "b"); m.put(1, "a");  
        System.out.println(m.keySet());  
    }  
}
```

Answer: [2, 1] **Explanation:** Insertion order retained.

Q90

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        Map<Integer,String> m = new TreeMap<>();  
        m.put(2, "b"); m.put(1, "a");  
        System.out.println(m.keySet());  
    }  
}
```

Answer: [1, 2]

Section 7: Streams and Functional Patterns (Q91-105)

Q91

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        int s = List.of(1,2,3).stream().mapToInt(Integer::intValue).sum();
        System.out.println(s);
    }
}
```

Answer: 6

Q92

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        System.out.println(List.of(1,2,3).stream().count());
    }
}
```

Answer: 3

Q93

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        System.out.println(List.of(1,2,3).stream().filter(x -> x > 5).findFirst().isPresent());
    }
}
```

Answer: false

Q94

```
import java.util.*;
import java.util.stream.*;
class Main {
    public static void main(String[] args) {
        System.out.println(Stream.of("a","bb","ccc").map(String::length).toList());
    }
}
```

Answer: [1, 2, 3]

Q95

```
import java.util.*;
class Main {
```

```
public static void main(String[] args) {  
    System.out.println(List.of(1,2,3,4).stream().reduce(1, (a,b) -> a*b));  
}  
}
```

Answer: 24

Q96

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(List.of(1,2,3).stream().anyMatch(x -> x % 2 == 0));  
    }  
}
```

Answer: true

Q97

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(List.of(2,4,6).stream().allMatch(x -> x % 2 == 0));  
    }  
}
```

Answer: true

Q98

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(List.of(1,2,3).stream().noneMatch(x -> x < 0));  
    }  
}
```

Answer: true

Q99

```
import java.util.*;  
import java.util.stream.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(Stream.of(1,2,2,3).distinct().toList());  
    }  
}
```

```
}  
}
```

Answer: [1, 2, 3]

Q100

```
import java.util.*;  
import java.util.stream.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(Stream.of(3,1,2).sorted().toList());  
    }  
}
```

Answer: [1, 2, 3]

Q101

```
import java.util.*;  
class Main {  
    public static void main(String[] args) {  
        String s = List.of("a", "b", "c").stream().reduce("", (x,y) -> x + y);  
        System.out.println(s);  
    }  
}
```

Answer: abc

Q102

```
import java.util.*;  
import java.util.stream.*;  
class Main {  
    public static void main(String[] args) {  
        System.out.println(Stream.of("go", "java").map(String::toUpperCase).toList());  
    }  
}
```

Answer: [GO, JAVA]

Q103

```
import java.util.*;  
import java.util.stream.*;  
class Main {  
    public static void main(String[] args) {  
        var r = Stream.of(1,2,3).peek(System.out::print).count();  
    }  
}
```

```
        System.out.println("-" + r);
    }
}
```

Answer: 123-3

Q104

```
import java.util.*;
import java.util.stream.*;
class Main {
    public static void main(String[] args) {
        System.out.println(Stream.of(1,2,3,4).skip(1).limit(2).toList());
    }
}
```

Answer: [2, 3]

Q105

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        int s = List.of(1,2,3).parallelStream().mapToInt(Integer::intValue).sum();
        System.out.println(s);
    }
}
```

Answer: 6

Section 8: Caches, Concurrent Maps, and Practical Patterns (Q106-120)

Q106

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> cache = new HashMap<>();
        cache.putIfAbsent("x", 1);
        cache.putIfAbsent("x", 2);
        System.out.println(cache.get("x"));
    }
}
```

Answer: 1

Q107

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> cache = new HashMap<>();
        cache.computeIfAbsent("x", k -> 10);
        cache.computeIfAbsent("x", k -> 20);
        System.out.println(cache.get("x"));
    }
}
```

Answer: 10

Q108

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> m = new HashMap<>();
        m.merge("k", 1, Integer::sum);
        m.merge("k", 2, Integer::sum);
        System.out.println(m.get("k"));
    }
}
```

Answer: 3

Q109

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> m = new HashMap<>();
        m.compute("a", (k,v) -> v == null ? 1 : v + 1);
        m.compute("a", (k,v) -> v == null ? 1 : v + 1);
        System.out.println(m.get("a"));
    }
}
```

Answer: 2

Q110

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String,Integer> m = new LinkedHashMap<>(16, 0.75f, true);
        m.put("a",1); m.put("b",2); m.put("c",3);
        m.get("a");
        System.out.println(m.keySet());
    }
}
```

```
}  
}
```

Answer: [b, c, a] **Explanation:** Access-order mode moves accessed key to end.

Q111

```
import java.util.*;  
class LRU<K,V> extends LinkedHashMap<K,V> {  
    private final int cap;  
    LRU(int cap) { super(16, 0.75f, true); this.cap = cap; }  
    protected boolean removeEldestEntry(Map.Entry<K,V> e) { return size() > cap; }  
}  
class Main {  
    public static void main(String[] args) {  
        LRU<Integer,Integer> l = new LRU<>(2);  
        l.put(1,1); l.put(2,2); l.put(3,3);  
        System.out.println(l.keySet());  
    }  
}
```

Answer: [2, 3]

Q112

```
import java.util.concurrent.*;  
class Main {  
    public static void main(String[] args) {  
        ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>();  
        m.computeIfAbsent("k", x -> 1);  
        System.out.println(m.get("k"));  
    }  
}
```

Answer: 1

Q113

```
import java.util.concurrent.*;  
class Main {  
    public static void main(String[] args) {  
        ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>();  
        try {  
            m.put("x", null);  
        } catch (NullPointerException e) {  
            System.out.println("NPE");  
        }  
    }  
}
```

Answer: NPE

Q114

```
import java.util.concurrent.*;
class Main {
    public static void main(String[] args) {
        ConcurrentHashMap<String,Integer> m = new ConcurrentHashMap<>();
        m.put("a", 1);
        m.forEach((k,v) -> {
            if (k.equals("a")) m.put("b", 2);
        });
        System.out.println(m.size() >= 1);
    }
}
```

Answer: true **Explanation:** Iteration is weakly consistent, no `ConcurrentModificationException` .

Q115

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
class Main {
    public static void main(String[] args) {
        AtomicInteger ai = new AtomicInteger(0);
        ai.incrementAndGet();
        ai.addAndGet(2);
        System.out.println(ai.get());
    }
}
```

Answer: 3

Q116

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
class Main {
    public static void main(String[] args) {
        LongAdder adder = new LongAdder();
        adder.increment();
        adder.add(4);
        System.out.println(adder.sum());
    }
}
```

Answer: 5

Q117

```
import java.util.concurrent.*;
class Main {
    public static void main(String[] args) throws Exception {
        CompletableFuture<Integer> f = CompletableFuture.supplyAsync(() -> 40)
            .thenApply(x -> x + 2);
        System.out.println(f.get());
    }
}
```

Answer: 42

Q118

```
import java.util.concurrent.*;
class Main {
    public static void main(String[] args) throws Exception {
        CompletableFuture<Integer> f = CompletableFuture.completedFuture(5)
            .thenCompose(x -> CompletableFuture.completedFuture(x * 2));
        System.out.println(f.get());
    }
}
```

Answer: 10

Q119

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
class Main {
    public static void main(String[] args) {
        ConcurrentHashMap<String, LongAdder> m = new ConcurrentHashMap<>();
        m.computeIfAbsent("hit", k -> new LongAdder()).increment();
        m.computeIfAbsent("hit", k -> new LongAdder()).increment();
        System.out.println(m.get("hit").sum());
    }
}
```

Answer: 2

Q120

```
import java.util.*;
class Main {
    public static void main(String[] args) {
        Map<String, Integer> m = new HashMap<>();
        m.put("a", 1);
        System.out.println(m.getOrDefault("b", -1));
    }
}
```

```
}  
}
```

Answer: -1

Interview Tips

- For Java output questions, explicitly mention compile-time vs runtime behavior.
 - Distinguish method overriding, method hiding, and field hiding.
 - Always call out string pool and wrapper caching when `==` appears.
 - Mention finally-return override whenever try/finally return appears.
 - In thread questions, mention determinism only when `join()` /ordering controls exist.
 - In virtual thread questions, mention Java 21+ and scheduling nondeterminism.
 - In collections questions, call out mutability (`List.of` , `Arrays.asList`) and iteration semantics.
 - In stream questions, call out laziness, terminal operations, and side-effect risks.
-

Companion: Regular Problems + Solutions (One-Stop)

For non-tricky, implementation-focused interview prep (Collections, Streams, Optional, Cache, Threads, Virtual Threads, Runnable, and DSA/Algo), use:

- `39_JAVA_ONE_STOP_DSA_ALGO_REGULAR_PROBLEMS.md`